

PARQUE CIENTÍFICO Y TECNOLÓGICO UTPL

Análisis de Solución del Reto

Plataforma SaaS de Distribución con Vendedores Freelance

Institución: Parque Científico y Tecnológico UTPL

Versión: 1.0

Fecha: Mayo 2026

Tecnología: Django (Python)

Documento generado en L^AT_EX PRESENTACION-1BIM

Contents

Capítulo 1: Introducción y Contexto 2

0.1 Introduccion 2

0.2 Problemática. 2

0.3 Metodología o Estrategia de Desarrollo 2

Capítulo 2: Planificación y Análisis 4

0.4 Análisis 4

0.5 Requisitos Funcionales 4

0.6 Requisitos No Funcionales 5

0.7 Historias de Usuario / Casos de Uso 6

0.7.1 Actores del Sistema 6

0.7.2 Casos de Uso - Módulo de Pedidos. 7

0.7.3 Historias de Usuario Representativas. 7

0.8 Sprint. 8

0.9 Diagramas: Clases y BD (Diseño Lógico)	8
0.9.1 Descripción del Diagrama de Clases	8
0.9.2 Preguntas Guía para el Diagrama de Clases y BD	9
0.10 Diagrama de Componentes.	9
0.11 Arquitectura Lógica y Física (Propuesta)	10
0.11.1 Arquitectura Lógica.	10
0.11.2 Arquitectura Física (Propuesta de Despliegue)	10
0.11.3 Flujo de Proceso del Pedido	11
0.12 Prototipo de Pantallas No Funcional.	11
Referencias	12

Capítulo 1: Introducción y Contexto

0.1 Introducción

El presente documento constituye el análisis formal de la solución propuesta para el reto planteado por el Parque Científico y Tecnológico de la Universidad Técnica Particular de Loja (UTPL). La solución aborda una problemática concreta de distribución n comercial en zonas rurales y semi-urbanas de Latinoamérica, mediante el diseño de una plataforma SaaS (Software as a Service) que conecta empresas proveedoras, vendedores freelance y tiendas o negocios destino.

Este informe documenta los requisitos funcionales y no funcionales identificados, los casos de uso, los diagramas preliminares y la arquitectura técnica propuesta, con especial énfasis en el uso del framework Django (Python) como tecnología principal de desarrollo backend.

0.2 Problemática

La brecha de abastecimiento que enfrentan tiendas y negocios en zonas rurales o semiurbanas de Latinoamérica constituye la problemática central del presente reto. Estos establecimientos presentan dificultades estructurales para acceder a proveedores mayoristas, originadas principalmente por:

Falta de conectividad a internet o inestabilidad de red o de banda ancha.

Barreras de alfabetización digital entre los propietarios y empleados de los negocios.

Ausencia de canales de distribución eficientes y formalizados.

Distancias geográficas que encarecen la logística tradicional.

Como consecuencia, muchos pequeños negocios dependen de intermediarios informales, pagan precios más altos o simplemente quedan desabastecidos de productos clave para su operación.

0.3 Metodología o Estrategia de Desarrollo

Para el desarrollo de la solución se adoptará la metodología ágil Scrum, organizada en sprints quincenales. Esta metodología permite entregas incrementales y verificaciones, facilitando la validación temprana de funcionalidades con los actores del sistema.

Herramientas y tecnologías de trabajo:

Compo- nente	Tecnología / Herramienta
Back-end	Django 5.x (Python 3.12) framework web de alto nivel
principal	
API	Django REST Framework (DRF) serializers, viewsets, routers
REST	
Base de	PostgreSQL 16 motor relacional con soporte ACID completo
datos	
Auten- ti- cación	JWT mediante djangorestframework-simplejwt + 2FA
Fron- tend	React 18 + Tailwind CSS SPA con vistas diferenciadas por rol
API	Nginx proxy inverso, rate limiting, terminación TLS
Gate- way	
Pasarela de	Integración con proveedor PCI-DSS (Stripe / PayPhone)
pagos	
Notifi- ca- ciones	Celery + Redis tareas asíncronas; SMTP / SendGrid para emails

Com- po- nente	Tecnología / Herramienta
Gestión de proyec- tos	GitHub Projects + tablero Kanban por sprint
Pro- toti- pado	Figma, wireframes y prototipo no funcional de pantallas
UI Con- trol de ver- siones	Git + GitHub (repositorio del equipo)
Docu- mentación API	drf-spectacular (OpenAPI 3 / Swagger UI)

Table 1: Stack tecnológico del proyecto

¿Por qué Django?

Django es un framework maduro y con baterías incluidas (batteries included) que ofrece: ORM potente compatible con PostgreSQL, sistema de autenticación y permisos integrado, panel de administración automático, soporte nativo para migraciones de base de datos y una comunidad activa. Su ecosistema con Django REST Framework permite construir APIs RESTful documentadas de manera eficiente, cumpliendo con los requisitos RF10 (API RESTful para integración ERP) y RNF08 (principios SOLID y cobertura de pruebas).

Capítulo 2: Planificación y Análisis

0.4 Análisis

La plataforma SaaS actúa como intermediaria inteligente entre tres actores principales:

Empresas Proveedoras (B2B): Registran sus productos, configuran reglas de venta y exámenes de certificación para vendedores.

Vendedores Freelance: Actúan como intermediarios físicos, visitan tiendas, levantan pedidos y obtienen comisiones por cada entrega confirmada.

Tiendas / Negocios destino: Beneficiarios finales que reciben el producto sin necesidad de interactuar directamente con la plataforma

digital.

El sistema también contempla un Administrador de Plataforma encargado de la gestión global de usuarios, planes de suscripción y configuración del sistema.

0.5 Requisitos Funcionales

Se identificaron 14 requisitos funcionales agrupados por módulo:

ID	Nombre	Descripción
RF01	Registro de Empresa Proveedora	El sistema permite el registro de empresas con validación de datos legales (RUC/SRI) y aprobación manual antes de activar la cuenta.
RF02	Gestión de Catálogo de Productos	Las empresas proveedoras pueden crear, editar y desactivar productos con precio, stock, categoría, imagen y límites mínimos de compra.
RF03	Registro de Vendedor Freelance	El sistema permite el registro de vendedores con datos personales, biometría opcional y estado de verificación pendiente.
RF04	Sistema de Certificaciones y Exámenes	Las empresas pueden crear exámenes de certificación; los vendedores deben aprobarlos para desbloquear categorías de productos restringidos.
RF05	Visualización de Comisiones	Antes de confirmar un pedido, el vendedor visualiza el desglose detallado de su comisión estimada (porcentaje, monto y condiciones).
RF06	Gestión de Pedidos con Anticipo	El sistema soporta el flujo de pedido con pago de anticipo (50% o 100%) obligatorio, bloqueando la generación del pedido si no se cumple.
RF07	Flujo de Pago Directo Empresa-Comprador	El dinero del cliente final se transfiere directamente a la empresa proveedora; la plataforma no retiene fondos del cliente.

Continúa en la siguiente página...

(Continúa de la página anterior)

ID	Nombre	Descripción
RF08	Liberación de Comisión por Entrega Confirmada	La comisión del vendedor se libera únicamente cuando la empresa proveedora confirma la entrega exitosa del pedido en la tienda destino.
RF09	Sistema de Calificación Bidireccional	Vendedores y empresas pueden calificarse mutuamente (1-5 estrellas con comentario) tras la confirmación de entrega de cada pedido.
RF10	API RESTful para Integración ERP	El sistema expone endpoints de API RESTful documentados (OpenAPI/Swagger) para integrar inventario, pedidos y estados de entrega con ERP externos.
RF11	Módulo Nativo de Inventario	Para empresas sin ERP, se ofrece un módulo nativo de gestión de inventario con alertas de stock bajo.
RF12	Panel de Control por Rol	Cada tipo de usuario (Empresa, Vendedor, Administrador) tiene un panel de control diferenciado con las funcionalidades correspondientes a su rol.
RF13	Gestión de Suscripciones y Facturación	El sistema gestiona planes de suscripción para empresas o calcula y cobra el fee por transacción (1-2%) de manera automática.
RF14	Notificaciones y Alertas	El sistema envía notificaciones (correo/plataforma) para eventos clave: pedido creado, anticipo recibido, entrega confirmada, comisión liberada y examen disponible.

0.6 Requisitos No Funcionales

Se identificaron 12 requisitos no funcionales:

ID	Categoría
RNF01	Desempeño
RNF02	Disponibilidad
RNF03	Escalabilidad

El rendimiento de las páginas principales cargan en menos de 3 segundos bajo condiciones normales. Consultas de catálogo e inventario responden en menos de 2 segundos.

La disponibilidad mínima del 99,5% mensual (máximo 3,6 horas de inactividad/mes).

La arquitectura de microservicios permite escalabilidad horizontal independiente de los módulos de Inventario, Pagos y Certificaciones ante picos de demanda.

(Continúa de la página anterior)

ID	Categoría	Descripción
RNF04	Seguridad	Autenticación con tokens JWT y soporte de 2FA para cuentas de empresa. Las sesiones expiran tras inactividad.

ID	Categoría	Descripción
RNF05	Seguridad Datos Financieros	Todos los datos de pago se transmiten bajo HTTPS/TLS 1.2+. No se almacenan datos de tarjeta en crudo; se delega a pasarela certificada PCI-DSS.
RNF06	Usabilidad (Vendedores)	Interfaz Mobile- rst. Botones mínimo 48×48px, fuentes no menores a 18px, flujos con máximo 3 pasos por acción principal.
RNF07	Usabilidad (Portal Empresarial)	Portal B2B optimizado para escritorio con barra lateral fija, tablas paginadas y exportación de datos (CSV/Excel).
RNF08	Mantenibilidad	El código sigue principios SOLID con cobertura de pruebas unitarias mínima del 70% en módulos críticos (Pagos, Certificaciones, Pedidos).
RNF09	Cumplimiento Normativo	El flujo de dinero está diseñado para cumplir regulaciones antilavado (AML) en Latinoamérica, garantizando que los fondos van directamente empresa-comprador.
RNF10	Compatibilidad	Compatible con Chrome, Firefox, Safari y Edge (últimas 2 versiones). Vista de vendedor responsive en dispositivos móviles.
RNF11	Contabilidad de Transacciones	La base de datos PostgreSQL garantiza propiedades ACID en todas las transacciones financieras, evitando registros inconsistentes o duplicados.
RNF12	Trazabilidad y Auditoría	Todas las acciones financieras quedan registradas en un log de auditoría inmutable con timestamp, usuario y estado.

0.7 Historias de Usuario / Casos de Uso

0.7.1 Actores del Sistema

Administrador de Plataforma: Gestiona usuarios, empresas, planes de suscripción y configuración global del sistema.

Empresa Proveedora: Gestiona su catálogo, con gura exámenes de certificación, con rma despachos y califica vendedores.

Vendedor Freelance: Se registra, rinde exámenes, genera pedidos en nombre de tiendas, paga anticipos y recibe comisiones.

0.7.2 Casos de Uso Módulo de Pedidos

ID	Caso de Uso	Actor Principal	Descripción
CU01	Crear Pedido	Vendedor Freelance	Selecciona productos del catálogo desbloqueado, especifica cantidades y tienda destino. El sistema calcula el anticipo y la comisión estimada.
CU02	Pagar Anticipo	Vendedor Freelance	Realiza el pago del anticipo (50% o 100%) mediante pasarela. El pedido pasa a En preparación solo si el pago es con rmado.
CU03	Confirmar Despacho	Empresa Proveedora	La empresa con rma la entrega exitosa en la tienda destino. El sistema libera automáticamente la comisión al vendedor.
CU04	Calificar	Vendedor / Empresa	Tras la confirmación, ambas partes pueden calificarse mutuamente (1 5 estrellas con comentario opcional).
CU05	Consultar Comisiones	Vendedor Freelance	El vendedor visualiza el historial de comisiones: pendientes, liberadas y proyectadas por pedidos activos.

0.7.3 Historias de Usuario Representativas

HU-01 (Vendedor): Como vendedor freelance, quiero ver el desglose de mi comisión antes de confirmar un pedido, para decidir si el margen justifica la visita a la tienda.

HU-02 (Empresa): Como empresa proveedora, quiero que la comisión del vendedor se libere automáticamente solo cuando yo con rme la entrega, para tener control sobre el pago y evitar fraudes.

HU-03 (Vendedor): Como vendedor freelance, quiero rendir un examen de certificación para desbloquear categorías de productos especiales y aumentar mi catálogo disponible.

HU-04 (Empresa): Como empresa proveedora, quiero integrar mi ERP con la plataforma v a API REST para sincronizar inventario y pedidos en tiempo real.

HU-05 (Admin): Como administrador, quiero revisar pedidos en disputa y tomar decisiones de resolución para garantizar la confianza en la plataforma.

0.8 Sprint

La planificación de sprints se distribuye en cuatro ciclos quincenales del primer bimestre:

Sprint	Objetivo	Entregables Principales
1	Configuración Sprintbase y autenticación	Proyecto Django configurado, modelos de usuario/empresa/vendedor, JWT auth, panel de admin básico.
2	Catálogo e Sprintinventario	CRUD de productos (RF02), módulo nativo de inventario (RF11), API endpoints para ERP (RF10).
3	Pedidos y pagos Sprint	Flujo completo de pedido (RF06), integración pasarela de pago, liberación de comisión (RF08), notificaciones (RF14).
4	Certificaciones, Sprintcalificaciones y panel de control	Sistema de exámenes (RF04), calificación bidireccional (RF09), dashboards por rol (RF12), pruebas y documentación.

Table 5: Planificación de sprints Primer Bimestre

0.9 Diagramas: Clases y BD (Diseño Lógico)

0.9.1 Descripción del Diagrama de Clases

El diagrama de clases del sistema re eja las entidades principales y sus relaciones. Las entidades clave son:

Usuario (base): id, email, password_hash, rol (admin / empresa / vendedor), fecha_creacion, activo.

EmpresaProveedora: hereda de Usuario. Agrega ruc, razon_social, estado_aprobacion, plan_suscripcion.

VendedorFreelance: hereda de Usuario. Agrega cedula, estado_verificacion, saldo_comision.

Producto: id, empresa_id (FK), nombre, precio, stock, categoria, activo, minimo_compra.

Pedido: id, vendedor_id (FK), estado (BORRADOR, PENDIENTE_PAGO, ANTICIPO_RECIBIDO, EN_PREPARACION, ENTREGADO, COMPLETADO, CANCELADO), anticipo_requerido, comision_estimada.

ItemPedido: pedido_id (FK), producto_id (FK), cantidad, precio_unitario.

Pago: id, pedido_id (FK), monto, tipo (anticipo/saldo), estado, referencia_pasarela.

Certificacion: id, empresa_id (FK), nombre, categoria_producto.

ResultadoExamen: vendedor_id (FK), certificacion_id (FK), puntaje, aprobado, fecha.

Calificacion: pedido_id (FK), calificador_id, calificado_id, estrellas (1 5), comentario.

0.9.2 Preguntas Guía para el Diagrama de Clases y BD

Las siguientes cinco preguntas orientan el diseño detallado del diagrama de clases y el modelo relacional, y serán respondidas formalmente al elaborar los diagramas en Figma y en el modelado lógico:

P1. Cómo se modela la herencia de roles de usuario (Admin, Empresa, Vendedor) en la base de datos relacional de PostgreSQL, usando Django ORM: tabla única con discriminador, tablas separadas con OneToOneField o MTI (Multi-Table Inheritance)?

Answer esperado: Evaluar la estrategia ptima en términos de integridad referencial, complejidad de consultas y compatibilidad con el sistema de autenticación de Django.

P2. Qué atributos y restricciones de integridad (constraints) son necesarios en la entidad Pedido para garantizar que un pedido no pueda avanzar de estado sin el anticipo con rmado, evitando inconsistencias de datos?

Answer esperado: Definir check constraints, transiciones de estado válidas y el rol de las propiedades ACID de PostgreSQL en este caso.

P3. Cómo se relacionan las entidades Certificación, Producto y VendedorFree-lance para implementar el desbloqueo de categorías de productos restringidos tras aprobar un examen?

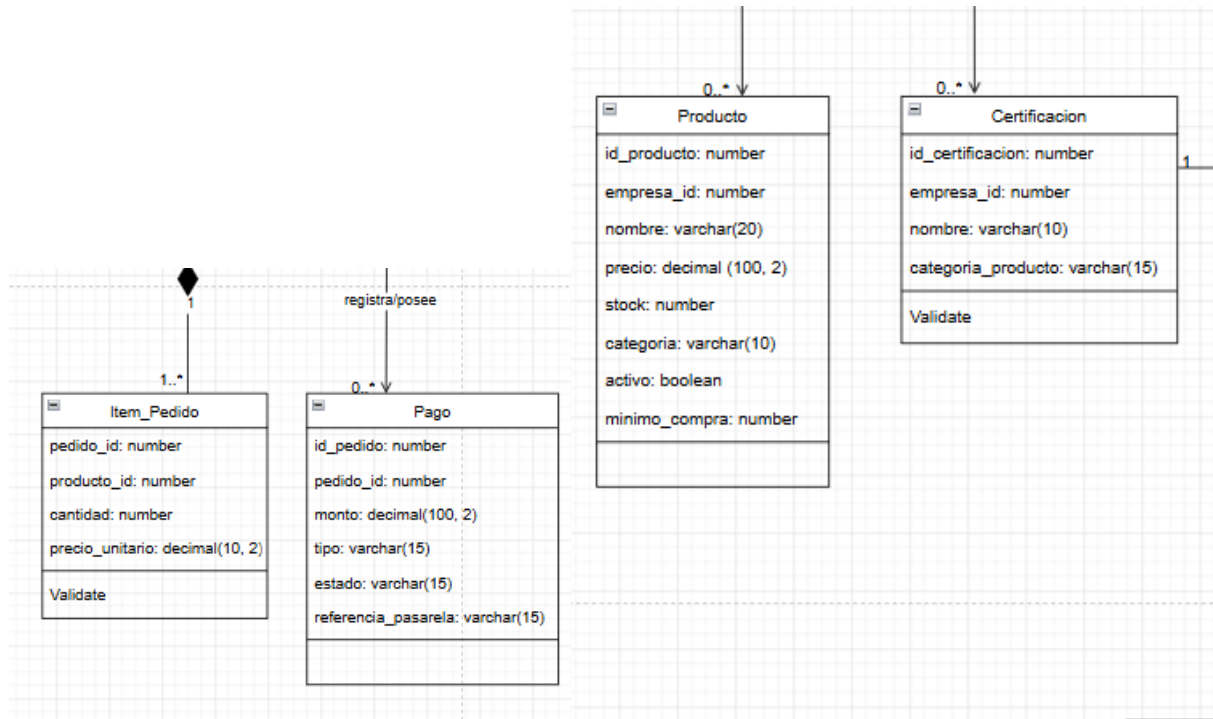
AnElisis esperado: Diseñar la tabla intermedia o relación many-to-many que permita consultar eficientemente qué productos puede vender un vendedor según sus certificaciones aprobadas.

P4. Cómo se garantiza la trazabilidad financiera inmutable de los eventos de pago, anticipo y comisión, cumpliendo con RNF12 (log de auditoría), sin impactar el rendimiento de las consultas transaccionales?

AnElisis esperado: Evaluar el uso de una tabla de auditoría separada con triggers o señales de Django, versus el uso de un log append-only, y su impacto en el diseño del diagrama de clases.

P5. Qué estructura de datos y relaciones se necesitan en el módulo de Calificación para soportar el sistema bidireccional (vendedor califica empresa y empresa califica vendedor) sin duplicar entidades ni generar ambigüedad en las referencias de `calificador_id` y `calificado_id`?

AnElisis esperado: Analizar el uso de Generic Relations de Django, claves foráneas polimórficas o un diseño con campo `tipo_calificacion` para distinguir la dirección de la calificación.



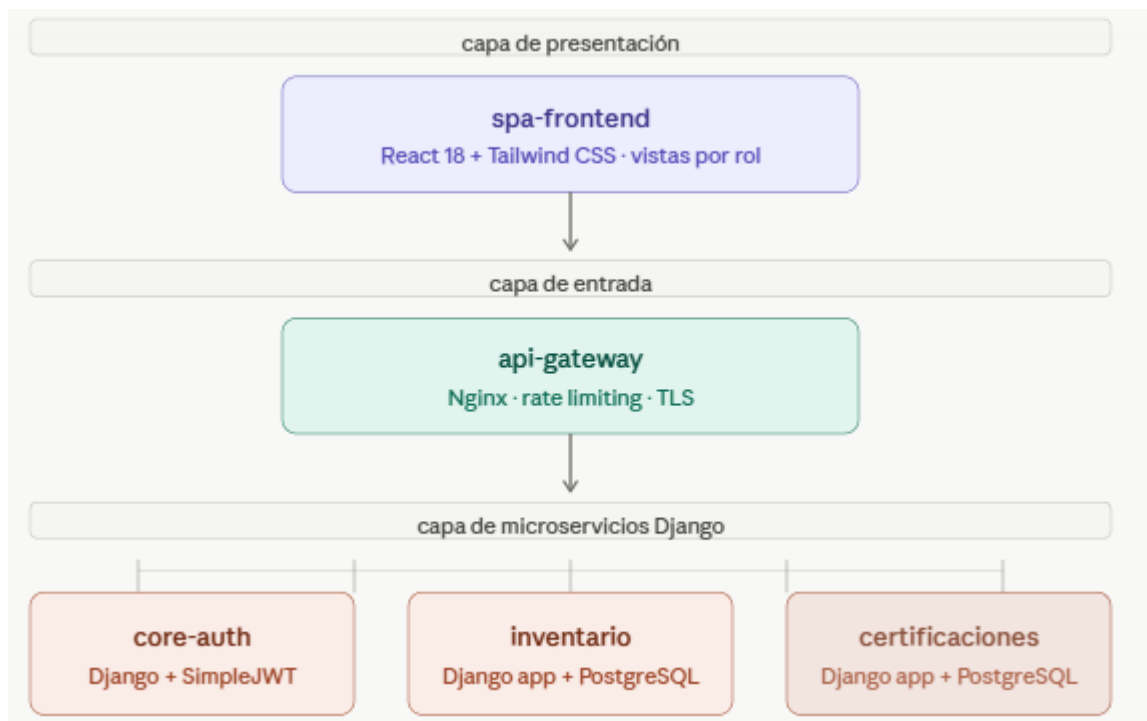
0.10 Diagrama de Componentes

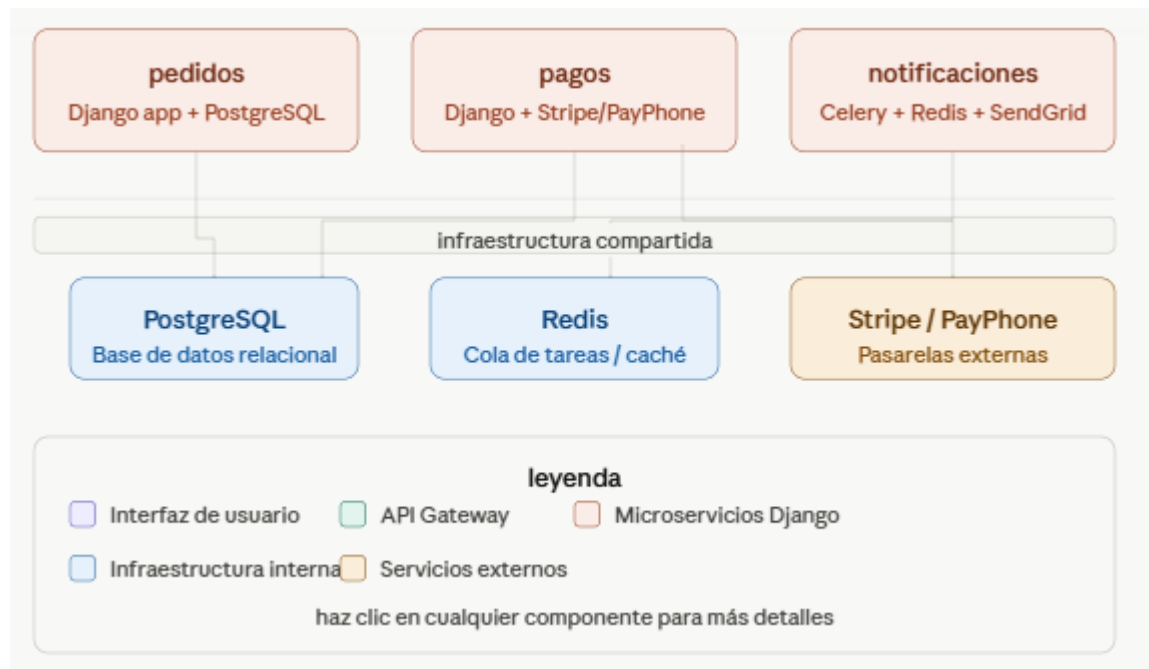
La arquitectura de componentes define los módulos del sistema y sus interfaces de comunicación:

Com- po- nente	Responsabilidad	Tecnología
core-auth	Autenticación, JWT, roles y permisos	Django + SimpleJWT
inventario	Catálogo de productos, stock, alertas	Django app + PostgreSQL
certificaciones	Exámenes, evaluaciones, desbloqueo de catálogo	Django app + PostgreSQL

Com- po- nente	Responsabilidad	Tecnolog a
pedi- dos	Ciclo de vida del pedido, estados, validaciones	Django app + PostgreSQL
pagos	Anticipos, comisiones, pasarela de pagos	Django app + Stripe/PayPhone
notifi- ca- ciones	Emails y alertas en plataforma	Celery + Redis + SendGrid
api- gateway	Enrutamiento, rate limiting, TLS	Nginx
spa- frontend	Interfaz web reactiva con vistas por rol	React 18 + Tailwind CSS

Table 6: Componentes del sistema y tecnolog as asociadas





0.11 Arquitectura Lógica y Física (Propuesta)

0.11.1 Arquitectura Lógica

La arquitectura lógica organiza el sistema en tres capas bien definidas:

1. Capa de Presentación (Cliente): SPA en React con vistas diferenciadas por rol (Administrador, Empresa, Vendedor). Diseño Mobile-irst para la vista de vendedor.
2. Capa de Negocio (API): Conjunto de aplicaciones Django expuestas v a Django REST Framework. Cada app Django corresponde a un dominio funcional (pedidos, pagos, inventario, etc.). La API Gateway (Nginx) centraliza el enrutamiento, el rate limiting y la terminación de TLS.
3. Capa de Datos: PostgreSQL 16 con esquema relacional normalizado. Propiedades ACID garantizadas para todas las transacciones financieras. CachØ con Redis para sesiones y cola de tareas asíncronas (Celery).

0.11.2 Arquitectura Física (Propuesta de Despliegue)

Servidor de aplicación: Contenedores Docker con Gunicorn (Django WSGI) detrás de Nginx.

Base de datos: Instancia PostgreSQL con réplica de lectura para reportes.

Broker de tareas: Redis para Celery (notificaciones asíncronas, liberación de comisiones).

CDN / Estáticos: Archivos estáticos y media servidos mediante CDN.

CI/CD: GitHub Actions con pipeline de pruebas automáticas y despliegue en entorno de staging.

0.11.3 Flujo de Proceso del Pedido

Paso	Estado	Descripción
1	BORRADOR	El vendedor selecciona productos, tienda destino y confirma el carrito.
2	PENDIENTE DE PAGO	Sistema calcula anticipo obligatorio. Vendedor recibe instrucciones de pago.
3	ANTICIPO RECIBIDO	Pasarela con rma el anticipo. Sistema notifica a la empresa.
4	EN PREPARACIÓN	Empresa prepara y despacha el pedido hacia la tienda destino.
5	ENTREGADO	Empresa con rma la entrega. Sistema libera automáticamente la comisión.
6	COMPLETADO	Ambas partes califican. Pedido archivado en historial.
	CANCELADO / EN DISPUTA	Si el pago falla o la entrega no se con rma en el plazo, el pedido entra en disputa con revisión del administrador.

Table 7: Estados del ciclo de vida del pedido

0.12 Prototipo de Pantallas No Funcional

El prototipo de pantallas no funcional será desarrollado en Figma y constituirá la base visual para el posterior diseño del diagrama de clases y la arquitectura de la base de datos. El prototipo contemplará las siguientes vistas:

Pantalla de inicio de sesión (roles diferenciados).

Dashboard del Administrador: gestión de empresas, vendedores y planes.

Dashboard de la Empresa Proveedora: catálogo de productos, pedidos activos, confirmación de despachos y calificaciones.

Dashboard del Vendedor Freelance (Mobile-first): catálogo disponible, creación de pedido, vista de comisiones y exámenes de certificación.

Flujo de creación de pedido: selección de productos cálculo de anticipo y comisión pago confirmación.

Módulo de exámenes: listado de certificaciones disponibles y pantalla de evaluación.

Nota: El enlace al prototipo en Figma será adjuntado en el repositorio de GitHub dentro de la carpeta PRESENTACION-1BIM, una vez finalizado el diseño de pantallas.

Referencias

Django Software Foundation. (2024). Django 5.x Documentation. [https:// docs.djangoproject.com/](https://docs.djangoproject.com/)

Encode OSS Ltd. (2024). Django REST Framework Documentation. <https://www.django-rest-framework.org/> PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/>

Parque Científico y Tecnológico UTPL. (2026). Análisis de Solución del Reto Plataforma SaaS de Distribución Freelance. Versión 1.0.